- ⚡ QUICKIDE 2.0 — Advanced Quantum Circuit IDE
  - 🧭 Overview
  - 🚀 The Three Pillars of QuickIDE 2.0
    - 1. Modern Developer Experience (IDE)
    - 2. Advanced Simulation Engine
    - 3. IBM Quantum Cloud Integration
  - 🛠️ Tech Stack
  - ⚙️ Quick Start (Development Mode)
    - 1. Database & Server (Node.js)
    - 2. Quantum Engine (Python API)
    - 3. Frontend (React)
  - 📚 QuCPL: Language at a Glance
  - 🤝 Contributing
  - 📄 License

⚡ QUICKIDE 2.0 — Advanced Quantum Circuit IDE

> 🧠 **Write. Debug. Simulate. Deploy.** — The modern fullstack Integrated Development Environment for the next generation of Quantum Developers.

🧭 Overview

QuickIDE 2.0 is a professional-grade quantum platform that transcends traditional educational tools. Completely rebuilt from the ground up, it combines a premium, high-performance web interface with an advanced quantum simulation engine powered by Qiskit.

Whether you are crafting intricate Bell states or deploying algorithms to real IBM Quantum hardware, QuickIDE 2.0 provides the tools, the aesthetics, and the depth required for industry-standard quantum development.

🚀 The Three Pillars of QuickIDE 2.0

1. Modern Developer Experience (IDE)

- 🎨 **Premium Aesthetic**: A dark-mode, glassmorphism UI built for long-duration coding sessions.
- 🧠 **Quantum IntelliSense**: Monaco-powered code editor with custom QuCPL syntax highlighting and gate-level autocomplete.
- 🐞 **Gate-Level Debugger**: The first IDE to offer a "Step-By-Step" quantum debugger. Inspect the exact **Statevector** and probability amplitudes at every single gate instruction.

2. Advanced Simulation Engine

- 🧪 **Hardware Noise Modeling**: Switch between ideal simulation and noisy hardware emulation (Manila, Nairobi) to understand real-world quantum decoherence.
- 🔁 **OpenQASM 3.0 Transpilation**: Automatic conversion from the high-level QuCPL language to industry-standard OpenQASM 3.0.
- 📊 **Rich Visualizations**: Real-time circuit diagrams, probability histograms, and statevector tables.

3. IBM Quantum Cloud Integration

- 🚀 **Real Hardware Submission**: Connect your IBM API Token to submit jobs directly to live quantum processors (e.g., `ibm_osaka`).
- ☁️ **Cloud Dashboard**: A dedicated workspace to manage your hardware job history, poll for results, and analyze multi-shot experiments.

🛠️ Tech Stack

QuickIDE 2.0 is a distributed fullstack application:

- **Frontend**: React.js with `allotment` for flexible IDE tiling.

- **Backend API**: Node.js & Express.
- **Quantum Core**: Flask (Python) with Qiskit SDK.
- **Database**: MongoDB (Project & Job persistence).
- **Authentication**: JWT-based secure session management.

⚙️ Quick Start (Development Mode)

QuickIDE 2.0 requires three services to run concurrently:

1. Database & Server (Node.js)

```
cd server
npm install
# Create .env with MONGO_URI and JWT_SECRET
npm run dev
```

2. Quantum Engine (Python API)

```
cd compiler_api
pip install -r ../requirements.txt
python app.py
```

3. Frontend (React)

```
cd client
npm install
npm start
```

📚 QuCPL: Language at a Glance

QuickIDE uses the **Quantum Circuit Programming Language (QuCPL)**, a high-level syntax designed for clarity:

```
// Create an entangled pair (Bell State)
qubit q0, q1;
qop h q0;
qop cx q0, q1;
measure q0, q1 -> c0, c1;
```

🤝 Contributing

Contributions are welcome! Please see our Developer Guide to get started with the QuCPL grammar and compiler logic.

📄 License

This project is licensed under the **MIT License**.

> *Crafted with precision to bring the power of the quantum cloud to your fingertips.*